

Instituto Federal de Educação, Ciência e Tecnologia da Bahia.

Docente: Felipe Oliveira.

Discente: Renata Santana dos Santos.

## **Sistema Operacional**

Um sistema operacional é um conjunto de programas que permite o gerenciamento dos recursos de hardware e software de um computador, fornecendo serviços essenciais para a execução de aplicativos, além de ser responsável por coordenar a interação entre o usuário, o hardware e o software.

Os componentes principais de um sistema operacional incluem o núcleo (kernel), que é a parte central responsável por gerenciar recursos e interagir diretamente com o hardware, o shell, que serve como a interface de interação do usuário com o sistema, e os sistemas de arquivos, que definem como os dados são organizados e acessados. Existem diferentes tipos de sistemas operacionais, como os de tempo real, que respondem a eventos com alta previsibilidade; monousuário e multiusuário, que variam no número de usuários simultâneos; de rede, que gerenciam redes de computadores; e os distribuídos, que permitem que múltiplos sistemas operacionais trabalhem em conjunto.

Os sistemas operacionais (SOs) são componentes essenciais na computação moderna, atuando como intermediários entre o usuário e o hardware do computador. Sua complexidade e funcionalidade exigem a aplicação de conceitos de raciocínio lógico em diversas áreas. Neste artigo, vamos explorar como o raciocínio lógico se interconecta com os sistemas operacionais, destacando suas implicações na tomada de decisões, controle de acesso, modelagem de processos, desenvolvimento de algoritmos e programação de sistemas.

### **A Relação entre Sistemas Operacionais e Raciocínio Lógico**

Um dos papéis fundamentais dos sistemas operacionais é a tomada de decisões em tempo real. O raciocínio lógico é crucial nesse contexto, especialmente no que diz respeito ao escalonamento de processos. Os sistemas operacionais

utilizam algoritmos de escalonamento para determinar qual processo deve receber tempo de CPU em um dado momento. Essas decisões são baseadas em condições lógicas, como prioridade do processo, tempo de espera e requisitos de recursos. Além disso, no gerenciamento de memória, os sistemas operacionais precisam decidir quando alocar ou liberar memória para processos, avaliando a disponibilidade de recursos e a necessidade de liberar memória de processos inativos. Essa capacidade de decisão lógica é essencial para garantir o funcionamento eficiente do sistema.

O raciocínio lógico desempenha um papel significativo na implementação de mecanismos de segurança em sistemas operacionais. Políticas de controle de acesso são baseadas em regras lógicas que determinam se um usuário ou processo tem permissão para acessar recursos específicos. Isso envolve verificações de identidade, permissões de arquivo e hierarquias de controle. Os algoritmos lógicos, por sua vez, são importantíssimos quando se trata de controle de acesso e segurança, uma vez que podem ser usados para detectar intrusões, analisando padrões de comportamento no sistema para identificar atividades suspeitas. Essa aplicação do raciocínio lógico é fundamental para proteger o sistema contra acessos não autorizados e garantir a integridade dos dados.

Os sistemas operacionais são fundamentais para o funcionamento eficiente dos computadores e dispositivos, atuando como intermediários entre o hardware e os usuários. Eles desempenham funções críticas de gerenciamento e controle de recursos, garantindo que os aplicativos possam ser executados de forma segura e eficiente.

## **A Relação entre Sistemas Operacionais e Análise de Complexidade**

Os sistemas operacionais desempenham um importante papel na gestão de recursos de hardware e software em um computador, e isso é possível graças à utilização da análise de complexidade, a qual envolve a avaliação do desempenho de algoritmos e sistemas em termos de tempo e espaço. Os sistemas operacionais gerenciam múltiplos processos simultaneamente, e a análise de complexidade é fundamental para avaliar a eficiência dos algoritmos de escalonamento, permitindo determinar o tempo médio de espera e o tempo de resposta para cada algoritmo de

escalonamento. Isso é crucial para otimizar o desempenho do sistema e garantir que os processos sejam executados de maneira eficiente.

A análise de complexidade também é importantíssima quando se trata de gerenciamento de memória, uma vez que os sistemas operacionais utilizam algoritmos para alocar e liberar memória, e a eficiência desses algoritmos pode ser analisada em termos de complexidade. Além disso, este tipo de análise ajuda a entender o impacto da fragmentação da memória e a eficácia das técnicas de compactação utilizadas pelos sistemas operacionais com o objetivo de minimizar problemas de consumo excessivo de recursos, por exemplo, CPU e memória, pelos processos em execução, melhorando a performance geral do sistema.

### Analizador:

Este analisador é um programa desenvolvido em C que tem como finalidade realizar uma análise detalhada de um arquivo de código-fonte. Sua principal tarefa é identificar a presença de subprogramas, como funções e procedimentos, além de detectar a utilização específica do algoritmo Merge Sort, ou seja, algoritmos de ordenação. Com base nessas informações, o programa não apenas classifica as estruturas presentes no código, mas também estima a complexidade computacional do algoritmo contido no arquivo analisado.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    FILE *arquivo;
    char linha[1000];

    arquivo = fopen("algoritmo.c", "r");
    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }

    printf("\n-----\n");

    int numeroDeLoopsFor = 0, numeroDeLoopsDoWhile = 0;
    int forAninhado = 0, doWhileAninhado = 0;
    int contagemSubprogramas = 0;
    int nivelAninhamentoFor = 0;
    bool isMergeSort = false;
```

Figura 1.

Na figura 1, há a inclusão das bibliotecas que serão utilizadas no analisador. Ainda nesta seção, o programa tenta abrir o arquivo "algoritmo.c" no modo de leitura, se a operação for bem-sucedida, o ponteiro "arquivo" irá referenciar o arquivo aberto, mas, se o arquivo não puder ser aberto, uma mensagem de erro é exibida.

```

while (fgets(linha, sizeof(linha), arquivo) != NULL) {
    printf("%s", linha);

    if (strstr(linha, "int main(") != NULL) {
    } else if (strstr(linha, "void ") != NULL ||
               strstr(linha, "int ") != NULL || strstr(linha, "float ") != NULL ||
               strstr(linha, "double ") != NULL || strstr(linha, "char ") != NULL ||
               strstr(linha, "bool ") != NULL) {

        if (strstr(linha, "(") != NULL) {
            contagemSubprogramas++;
        }
    }

    if (strstr(linha, "mergeSort") != NULL) {
        isMergeSort = true;
    }

    if (strstr(linha, "for(") != NULL) {
        numeroDeLoopsFor++;

        if (numeroDeLoopsFor > 1) {
            forAninhado = numeroDeLoopsFor;
        }
    }

    if (strstr(linha, "do {") != NULL || strstr(linha, "while(") != NULL) {
        numeroDeLoopsDoWhile++;
        if (numeroDeLoopsDoWhile > 1) {
            doWhileAninhado = numeroDeLoopsDoWhile;
        }
    }
}

fclose(arquivo);

```

**Figura 2.**

Esse trecho de código lê um arquivo de texto linha por linha, utilizando a função `fgets`, e imprime cada linha no console. Durante a leitura, ele verifica se a linha contém a declaração da função principal `int main()` e a ignora, pois a função `main` não é considerada um subprograma. Se a linha incluir tipos de subprogramas, o código incrementa um contador correspondente. Além disso, ele verifica a presença do algoritmo Merge Sort, definindo uma flag para indicar sua ocorrência. O código também conta o número de loops `for` e `do while/while`, permitindo identificar se há aninhamento desses laços de repetição, e atualiza variáveis que registram esse alinhamento, para que seja possível exibir a análise de complexidade do algoritmo informado. Após concluir a leitura e análise, o código fecha o arquivo para liberar os recursos utilizados.

```

printf("\n-----\n");
if (isMergeSort) {
    printf("Nível de complexidade do algoritmo: O(n log n)\n");
} else if (forAninhado == 1 || doWhileAninhado == 1) {
    printf("Nível de complexidade do algoritmo: O(n^%i)\n", forAninhado);
} else if (forAninhado == 0 || doWhileAninhado == 0) {
    printf("Nível de complexidade do algoritmo: O(n)\n");
}

printf("Quantidade de subprogramas: %i \n", contagemSubprogramas);

return 0;

```

**Figura 3.**

Esse trecho de código faz a exibição do resultado da análise de complexidade do algoritmo, utilizando como parâmetro as variáveis de controle utilizadas.

## Referências Bibliográficas

<https://tecnoblog.net/responde/o-que-e-um-sistema-operacional/>. Acesso em: 10 out. 2024.

<https://www.canalti.com.br/sistemas-operacionais/sistema-operacional-fundamentos-funcionamento-e-tipos-de-so-informatica-basica/> . Acesso em: 10 out. 2024.

[Sistemas Operacionais: Conceitos e Mecanismos \(ufpr.br\)](#) . Acesso em: 12 out. 2024.

<https://www.iugu.com/blog/analise-complexidade-algoritmos> . Acesso em: 12 out. 2024.

<https://agostinhobritojr.github.io/curso/algoritmos/so.pdf> . Acesso em: 12 out. 2024.